Sakana Fugu Ultra × Codex CLI
Long-Horizon Agent Behavior Report

Codex local research packet

experiments/2026-05-14-sakana-fugu-codex-cli-agent-behavior

2026-05-14

Abstract

I analyze Sakana AI `fugu-ultra high` running a long Project II Phase II validation task inside OpenAI Codex CLI v0.130.0. My central conclusion is that Fugu Ultra did not fail because it lacked technical exploration ability; it failed as a long-horizon agent because it did not maintain enough state discipline. It explored repositories, Docker and IC setup, ELF properties, debugger evidence, coredumps, libc and gadget paths, and bounded sweeps. The failure point was state compression, negative-evidence preservation, convergence control, token budget management, and proof-boundary discipline. I compare the Fugu run with an Ubuntu GPT-5.5 handoff run and a Mac ARM64 GPT-5.5 completion attempt. I do not claim GPT-5.5 proved better exploit-solving ability. I claim that GPT-5.5 left a cleaner, more reproducible, more honest continuation state.

Contents

1 Abstract and Introduction

I write this report in the first person because I am making a defensible research judgment from observable logs, artifacts, token usage, git state, and validation boundaries. I separate verified facts from interpretation, and I state clearly which claims I do not make.

My core position is: **I do not equate failure to solve the exploit with absence of model capability; I also do not equate large reasoning-token volume with task progress. I evaluate whether a long-horizon agent can convert exploration into durable state, negative evidence, stop rules, handoff artifacts, and an honest completion boundary.**

The effective Project II Phase II success condition is narrow:

```
The official IC Phase II workflow must produce /shared/success.txt.
```

I reject EC-side success-file creation, manual `/backdoor` execution, stale artifacts, coredump-only evidence, and scaffold-only checks as full-credit proof. This proof boundary is the foundation of the report.

2 Evidence Sources

Evidence type	Copied log	How I use it
Primary Fugu behavior log	`source-logs/2026-05-13-ubuntu-fugu-ultra-phase2-validation-transcript.md`	I analyze token accumulation, re-exploration, state drift, and unfinished official success.
GPT-5.5 handoff contrast	`source-logs/2026-05-13-ubuntu-gpt55-phase2-handoff-source.md`	I compare state compression and verification-first handoff behavior.
GPT-5.5 Mac validation contrast	`source-logs/2026-05-14-mac-gpt55-git-publish-source.md`	I compare host recovery, source-of-truth negative validation, artifact preservation, and git closeout.

These logs are not a controlled A/B benchmark. I treat them as trajectory forensics: evidence for studying state discipline, context engineering, proof boundaries, negative evidence, and repository hygiene.

3 Executive Finding

My main conclusion is:

> In this Codex CLI setting, `fugu-ultra` did not fail because it was incapable of technical exploration. It failed as a long-horizon agent because it did not maintain enough state discipline.

Fugu Ultra performed real work: repository search, lab extraction, Docker/IC startup, ELF and gadget inspection, coredump validation, libc and one-gadget exploration, bounded sweeps, documentation updates, and static checks. The problem was not a lack of exploratory ability. The problem was that high-value findings were not compressed early enough into stable task state, failed-hypothesis ledgers, and proof-boundary gates.

The strongest evidence is the token/outcome contrast:

Metric	Fugu value	Interpretation
total tokens	6,319,482	Very high cost for an unfinished outcome.
input tokens	6,136,936	Active input pressure was extreme.
cached input	23,226,988	Long context was repeatedly reused.
output tokens	182,546	Large visible output did not become official completion.
reasoning tokens	3,437,721	Reasoning cost did not convert into the required proof artifact.
official success	not observed	No IC-side `/shared/success.txt`.

4 Three-Run Comparison

Dimension	Ubuntu / Fugu Ultra	Ubuntu / GPT-5.5 handoff	Mac ARM64 / GPT-5.5 completion
Main task	Direct Phase II completion	Handoff generation	Completion attempt + environment repair
Host friction	Low	Low	High
Technical exploration	High	Medium-low	High
State compression	Low	Very high	High
Negative evidence preservation	Medium-low	High	Very high
Honest completion boundary	Medium	High	Very high
Continuation readiness	Medium-low	Very high	Very high
Official `/shared/success.txt`	No	No	No
Total tokens	6,319,482	not shown	1,098,226
Reasoning tokens	3,437,721	not shown	52,062
Closeout state	Local modifications / incomplete closure	Handoff artifact	Commit `ab319c0`, pushed to main, clean worktree

My interpretation is direct: Fugu Ultra explored deeply, but it did not convert exploration into a stable decision system quickly enough. GPT-5.5 did not solve the exploit either, but it produced a cleaner engineering state.

5 Mac ARM64 Run: Environment Repair as Evidence

The Mac run strengthens my conclusion because it was a harder host environment. The log verifies MacBook-Air, macOS, ARM64, and `ARM64_T8112`. I therefore use "Mac Air ARM64 host" as the rigorous phrasing; I do not treat "M1" as log-verified hardware identity inside the report.

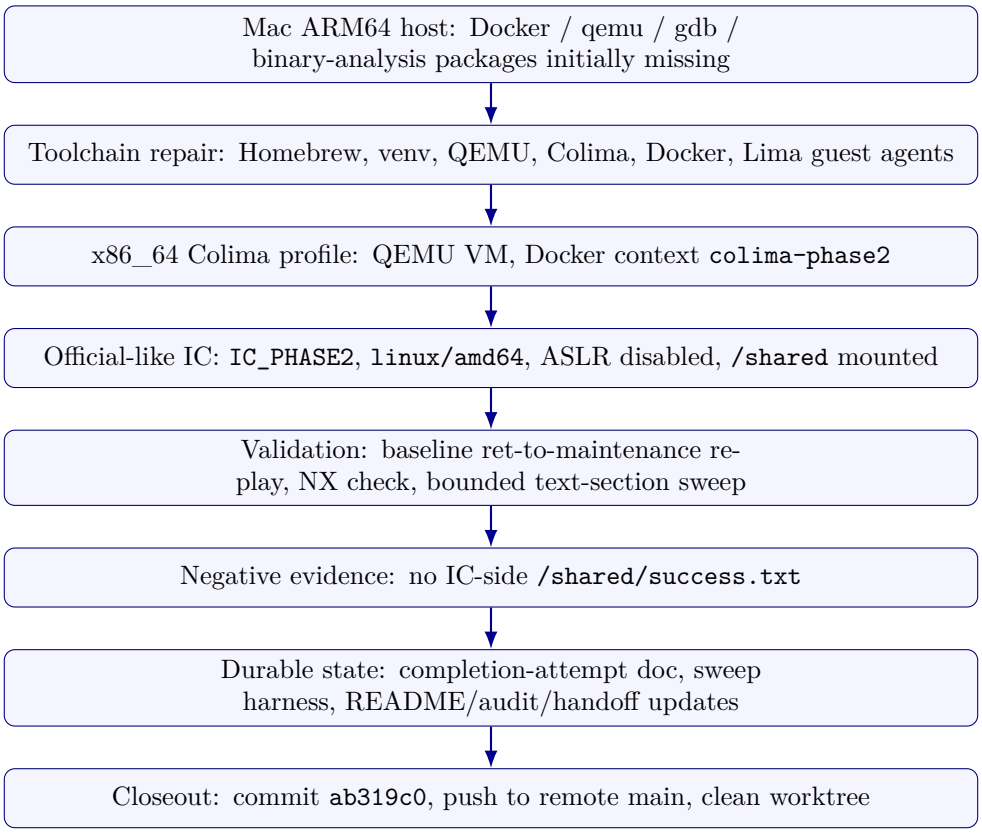

Figure 1: Mac ARM64 run as environment repair plus validation-state compression.

I consider this a meaningful agent-behavior result. GPT-5.5 turned host friction into a reproducible validation path instead of treating the missing environment as either success or a terminal blocker.

6 Token and Reasoning Cost

I chart only the two completion/validation runs with complete visible token usage. The Ubuntu GPT-5.5 handoff log did not expose token usage, so I exclude it from the numeric chart.

Metric	Ubuntu Fugu Ultra	Mac GPT-5.5	My interpretation
Total tokens	6,319,482	1,098,226	Fugu used about 5.75x more total tokens.
Reasoning tokens	3,437,721	52,062	Fugu used about 66x more reasoning tokens.
Cached input	23,226,988	28,131,328	High cached context alone did not cause reasoning collapse.
Reasoning / total	54.4%	4.7%	Fugu concentrated cost in active reasoning.
Reasoning / output	18.8:1	0.48:1	Fugu converted reasoning into artifacts less efficiently.

This is not a fully fair model benchmark because the Mac run had a prior handoff. Still, I find the behavior difference important: Fugu's reasoning expanded without closure, while GPT-5.5 kept reasoning more tightly coupled to decisions and artifacts.

Figure 2: Token cost comparison for the two runs with complete visible token usage.

7 Technical Negative Evidence

The Mac run did not solve Phase II. Its value is that it made several plausible paths explicitly non-repeatable:

Path tested	Result	Why I consider it useful
Current ret-to-`maintenance_task+5` candidate	`success_exists=no`; no `/shared/success.txt`.	The candidate remains non-success; argument control is still unresolved.
Direct stack shellcode	Reached stack address but faulted under NX.	Stack-resident shellcode is excluded in the live runtime.
One-shot text-section partial return	`0x401000..0x401a20`, four prefixes, 10,328 candidates, no success.	The next agent should not rerun the same blind landing-point sweep.
Manual `/backdoor`	Sanity check only.	It is not official proof.
Colima/QEMU IC setup	Reproducible `x86_64 linux/amd64` path.	Future runs should reuse this path rather than rebuilding the environment from scratch.

My current technical conclusion is:

```
The core blocker is no longer "we have not confirmed overflow,"
"we have not confirmed the success condition," or
"we have not reproduced an official-like IC runtime."

The core blocker is:
how to obtain reliable pivot and first-argument control under
C-string / NUL-byte constraints.
```

8 Behavior Scoring

These scores describe behavior profiles, not absolute model intelligence.

Evaluation dimension	Fugu Ubuntu	GPT-5.5 Ubuntu handoff	GPT-5.5 Mac ARM64 completion
Task understanding	3.5/5	5/5	5/5
Environment understanding	4/5	4/5	5/5
Environment repair	3/5	N/A	5/5
Technical exploration	4.5/5	3/5	4/5
Dynamic validation	3/5	2/5	4.5/5
State compression	1.5/5	5/5	4.5/5
Negative evidence quality	2/5	3/5	5/5
Token efficiency	1/5	High, but no token count	3.5/5
Completion-boundary honesty	3/5	5/5	5/5
Repository hygiene	2.5/5	4/5	5/5
Final exploit success	0/5	N/A	0/5

My conclusion from this table is not that GPT-5.5 solved the exploit. It did not. My conclusion is that GPT-5.5 managed the unsolved state more professionally.

9 Claims I Make and Claims I Reject

I conclude:

- Fugu Ultra showed real technical exploration ability.
- Fugu Ultra's main failure mode was long-horizon state discipline: token amplification, memory fragmentation, delayed compression, and weak convergence control.
- GPT-5.5 showed stronger context engineering in the handoff run.
- GPT-5.5 showed stronger host-environment recovery and negative-evidence preservation in the Mac run.
- The repository itself should be treated as the durable memory system for long-running agents.

I do not conclude:

- I do not conclude that GPT-5.5 is proven to be better at solving this exploit.
- I do not conclude that Fugu Ultra lacks technical ability.
- I do not conclude that Codex CLI hierarchical memory alone caused the failure.
- I do not conclude that the Mac run achieved full-credit Project II completion.

The sharper claim is:

> Fugu Ultra lost control of long-horizon state; GPT-5.5 preserved state better. The decisive difference in these logs is not solved-vs-unsolved exploit outcome, but the quality of the unfinished state left behind.

10 Next Fair Experiment Matrix

Group	Host	Model	Starting point	Purpose
A	Ubuntu 24	Fugu Ultra	`HANDOFF_PHASE2.md` + Mac negative evidence	Test whether external memory reduces Fugu drift.
B	Ubuntu 24	GPT-5.5	Mac 2026-05-14 artifacts	Test whether GPT can exploit the new negative evidence.
C	Mac ARM64	Fugu Ultra	Same handoff + existing Colima runbook	Test Fugu host recovery and state discipline.
D	Mac ARM64	GPT-5.5	Already-built Colima IC	Isolate exploit reasoning from environment bootstrap.

The key metrics should be official success rate, false-completion rate, tokens per useful evidence item, duplicate-command rate, failed-hypothesis retry rate, no-new-evidence streak, artifact quality, repo hygiene, and time-to-official-validation.

11 Final Conclusion

My final conclusion is:

> Fugu Ultra in the Ubuntu native run showed strong exploration but weak convergence. GPT-5.5 in the Ubuntu handoff run performed state compression. GPT-5.5 in the Mac ARM64 run, despite a harder host environment, built an x86_64 Colima validation path, produced deeper negative evidence, preserved a reproducible sweep harness, updated the repository, and pushed a clean commit.

I keep the boundary explicit:

> GPT-5.5 did not complete Project II full-credit success. It completed a cleaner, more reproducible, more honest engineering-validation loop.

That distinction is the research value of this packet. In an agent-system evaluation, the strongest evidence is not who solved the exploit. No run solved it. The strongest evidence is what each agent left behind when it failed:

```
Fugu Ultra left a high-token, low-convergence long trace.
GPT-5.5 left durable docs, a harness, a pushed commit, and negative evidence.
```

That is the difference I would carry into the next agent benchmark design.